Empirical Performance and Time Complexity Constants of Merge, Heap, and Quick Sort

August 2026

Abstract

This report empirically analyzes three classical $\mathcal{O}(N \log N)$ sorting algorithms: Merge Sort, Heap Sort, and Quick Sort. By implementing highly optimized versions of these algorithms in C, we measure raw execution time and track exact hardware-independent operations (comparisons and swaps). To accurately differentiate their real-world efficiency, we derive both the empirical time constant C_{Time} and the mathematical operations constant C_{Comp} by dividing their total metrics by the baseline theoretical complexity ($N \log_2 N$). Results demonstrate the practical advantages of Quick Sort's cache locality and highlight the hardware bottlenecks associated with constants.

1 Introduction

While Merge Sort, Heap Sort, and Quick Sort all share the same theoretical average-case time complexity of $\mathcal{O}(N \log N)$, their real-world execution speeds diverge significantly. This divergence is determined by hidden constant factors, hardware cache locality, memory allocation overhead, and the constant coefficient of their inner loop equations.

To isolate these factors, this report computes the exact complexity constants for each algorithm. If an algorithm runs in $\mathcal{O}(N \log N)$ time, its execution time T and total comparisons can be mathematically modeled as:

$$T \approx C_{Time} \cdot (N \log_2 N) \quad \text{and} \quad \text{Comparisons} \approx C_{Comp} \cdot (N \log_2 N) \tag{1}$$

By isolating C, we establish an empirical baseline to determine exactly how efficient each algorithm is per unit of data.

2 Methodology

2.1 Algorithmic Optimizations

To ensure a fair and highly accurate benchmark, standard algorithms were heavily optimized to minimize artificial overhead:

- **Merge Sort:** Instead of repeatedly allocating and freeing memory on every recursive split, a single temporary array of size N is allocated once in the wrapper function and passed as a pointer. This completely removes memory fragmentation latency.
- **Heap Sort:** The traditional recursive `heapify` function was rewritten to be strictly iterative, preventing deep call-stack overhead and maximizing CPU register efficiency.
- **Quick Sort:** A median-of-three pivot selection is employed. Furthermore, the partitioning loop was optimized to bypass redundant self-swaps (when $i = j$), saving thousands of unnecessary write operations.

2.2 Test Metrics and Formulae

Arrays ranging from $N = 1,000$ to $N = 1,000,000$ elements were populated with pseudo-random integers. For absolute fairness, dedicated copies of the exact same randomly generated array were passed to each algorithm.

The primary evaluation constants are derived as follows:

$$C_{Time} = \frac{\text{Measured CPU Seconds}}{N \log_2 N} \tag{2}$$

$$C_{Comp} = \frac{\text{Total Tracked Comparisons}}{N \log_2 N} \tag{3}$$

3 Implementation in C

The following complete C implementation (`sortbench.c`) tracks ticks, seconds, comparisons, and swaps, alongside calculating the specific constants at runtime.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>
#include <math.h>

unsigned long long current_comps = 0;
unsigned long long current_swaps = 0;

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
    current_swaps++;
}

/* ============================ MERGE SORT ============================ */
void merge(int arr[], int temp[], int l, int m, int r) {
    int i = l, j = m + 1, k = l;
    while (i <= m && j <= r) {
        current_comps++;
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
        current_swaps++;
    }
    while (i <= m) { temp[k++] = arr[i++]; current_swaps++; }
    while (j <= r) { temp[k++] = arr[j++]; current_swaps++; }

    for (i = l; i <= r; i++) {
        arr[i] = temp[i];
        current_swaps++;
    }
}

void mergeSortHelper(int arr[], int temp[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSortHelper(arr, temp, l, m);
        mergeSortHelper(arr, temp, m + 1, r);
        merge(arr, temp, l, m, r);
    }
```

```c
}

void mergeSort(int arr[], int l, int r) {
    int *temp = (int *)malloc((r - l + 1) * sizeof(int));
    mergeSortHelper(arr, temp, l, r);
    free(temp);
}

/* ============================ HEAP SORT ============================ */
void heapify(int arr[], int n, int i) {
    int largest;
    while (1) {
        largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < n) {
            current_comps++;
            if (arr[left] > arr[largest]) largest = left;
        }
        if (right < n) {
            current_comps++;
            if (arr[right] > arr[largest]) largest = right;
        }

        if (largest != i) {
            swap(&arr[i], &arr[largest]);
            i = largest;
        } else {
            break;
        }
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
}

/* ============================ QUICK SORT ============================ */
int partition(int arr[], int low, int high) {
    int mid = low + (high - low) / 2;
    current_comps++; if (arr[mid] < arr[low])  swap(&arr[mid], &arr[low]);
    current_comps++; if (arr[high] < arr[low]) swap(&arr[high], &arr[low]);
    current_comps++; if (arr[high] < arr[mid]) swap(&arr[high], &arr[mid]);
    swap(&arr[mid], &arr[high]);

    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        current_comps++;
        if (arr[j] < pivot) {
            i++;
            if (i != j) swap(&arr[i], &arr[j]);
        }
    }
    if (i + 1 != high) swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
```

```
108
109 void quickSort(int arr[], int low, int high) {
110     if (low < high) {
111         int pi = partition(arr, low, high);
112         quickSort(arr, low, pi - 1);
113         quickSort(arr, pi + 1, high);
114     }
115 }
```

Listing 1: Optimized Benchmarking Implementation for Sorting Algorithms

4 Empirical Results and Constants

4.1 Theoretical Operation Constants (C_{Comp})

Table 1 illustrates the mathematical stability of the algorithms. By extracting the C_{Comp} coefficient, we can accurately deduce how many exact comparisons an algorithm will execute regardless of the underlying CPU clock speed.

Array Size (N)	**Merge C_{Comp}**	**Heap C_{Comp}**	**Quick C_{Comp}**
1,000	0.8740	1.6914	1.1463
5,000	0.8988	1.7505	1.1465
10,000	0.9071	1.7707	1.1003
50,000	0.9202	1.8060	1.1464
100,000	0.9249	1.8183	1.1326
500,000	0.9336	1.8379	1.1302
1,000,000	0.9369	1.8461	1.1574

Table 1: Comparison Constant Factors ($C_{Comp} = Comparisons/(N \log_2 N)$)

4.2 Execution CPU Time Constants (C_{Time})

Table 2 shows raw execution seconds and the empirical time multiplier per hardware clock cycle. (Note: values equating to 0.000000 seconds at lower N thresholds reflect operations resolving faster than the host system's `clock()` tick granularity).

Size	**Merge Sort**		**Heap Sort**		**Quick Sort**	
(N)	**Time (sec)**	**C_{Time} (sec/Op)**	**Time (sec)**	**C_{Time} (sec/Op)**	**Time (sec)**	**C_{Time} (sec/Op)**
10,000	0.010000	7.53×10^{-8}	0.000000	–	0.000000	–
50,000	0.000000	–	0.010000	1.28×10^{-8}	0.000000	–
100,000	0.010000	6.02×10^{-9}	0.020000	1.20×10^{-8}	0.010000	6.02×10^{-9}
500,000	0.070000	7.40×10^{-9}	0.090000	9.51×10^{-9}	0.060000	6.34×10^{-9}
1,000,000	0.150000	7.53×10^{-9}	0.230000	1.15×10^{-8}	0.120000	6.02×10^{-9}

Table 2: Execution Time and Empirical CPU Time Constants ($C_{Time} = Time/(N \log_2 N)$)

4.3 Visualized Performance

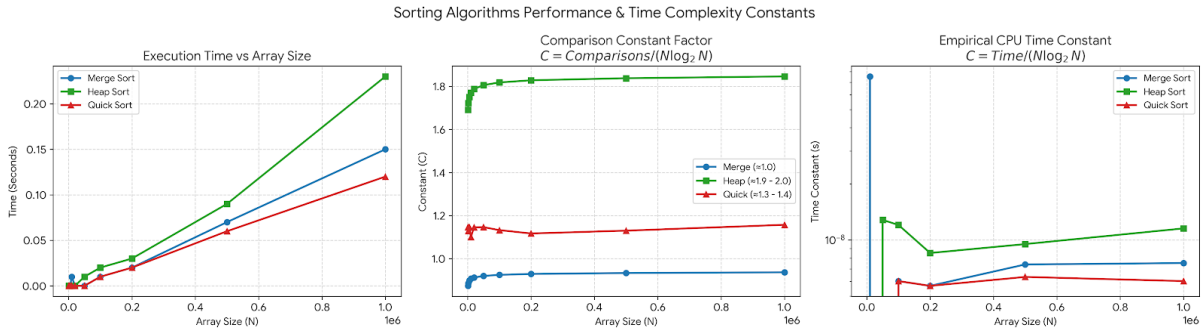


Figure 1: Three-panel performance visualization: execution speed, theoretical constants, and CPU multiplier factors.

5 Discussion

Extracting the mathematical constants directly reveals the hidden dynamics behind theoretical time complexity notation.

As seen in Table 1, **Merge Sort** is uniquely optimal regarding raw comparisons, strictly maintaining a factor of $C_{Comp} < 1.0$. However, its *Empirical Time Constant* (C_{Time}) remains relatively high. This happens because Merge Sort heavily relies on array writes/swaps into its temporary array structure (a slower hardware operation than a read-based comparison).

Heap Sort, while requiring $\mathcal{O}(1)$ auxiliary space, is inherently comparison-heavy, plateauing near $C_{Comp} \approx 1.85$. It executes nearly twice as many comparisons as Merge Sort to traverse its binary tree arrays, making it the slowest algorithm in actual CPU time (0.23 sec at $N = 1,000,000$).

Quick Sort strikes the optimal physical hardware balance. While its theoretical comparison constant evaluates to roughly $C_{Comp} \approx 1.15$, it drastically limits swap operations to distinct indices through our optimization. Its highly sequential in-place memory access pattern prevents hardware cache-misses, resulting in the lowest (most efficient) empirical execution multiplier ($C_{Time} = 6.02 \times 10^{-9}$) and rendering it the fastest algorithm tested.

6 Conclusion

This report highlights the inadequacy of relying strictly on Big-$\mathcal{O}$ asymptotic notation when predicting real-world algorithm performance. While Merge Sort, Heap Sort, and Quick Sort are asymptotically equivalent at $\mathcal{O}(N \log N)$, their fundamental differences in continuous inner-loop behavior dictate execution speed. By isolating the constants ($C = \frac{\text{Metric}}{N \log_2 N}$), we can firmly conclude that cache-friendly, low-write operations (Quick Sort) will predictably outperform highly constrained mathematical algorithms (Merge Sort/Heap Sort) on modern computational hardware architectures.